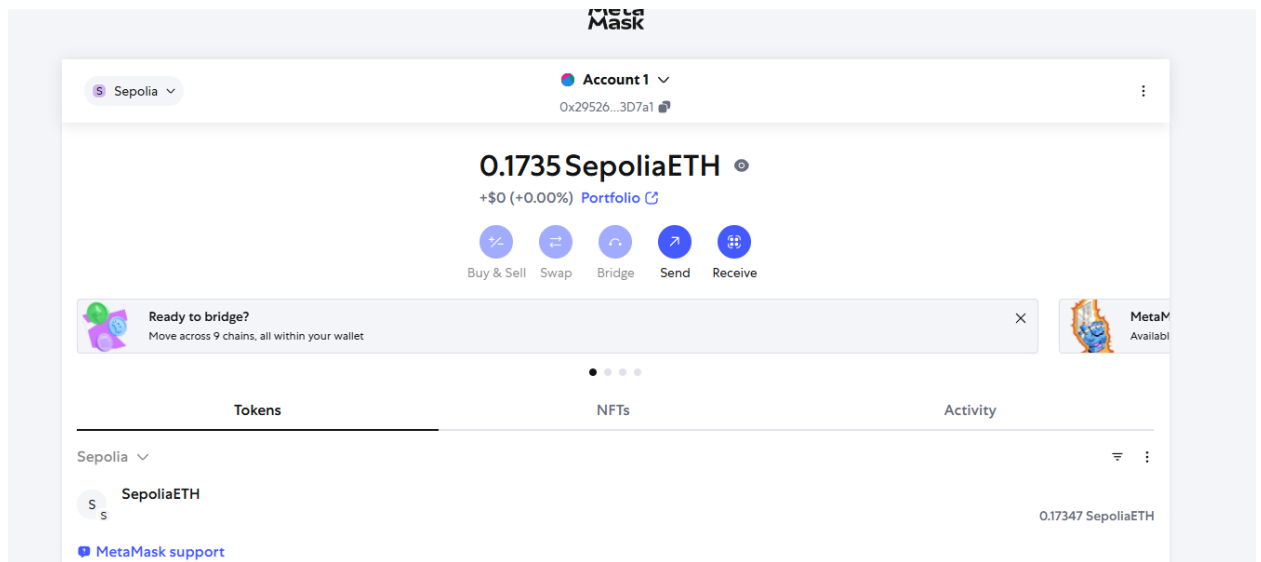


Limpin,Jholim Jats R.
NW-301

Part 1 — Setup: MetaMask Sepolia + Test ETH

Task 1.1: Enable Sepolia in MetaMask

1. Open MetaMask.



2. Go to **Settings** → **Advanced**.
3. Turn ON **Show test networks**.

Show conversion on test networks

Select this to show fiat conversion on test networks

☐ OFF

Show test networks

Select this to show test networks in network list

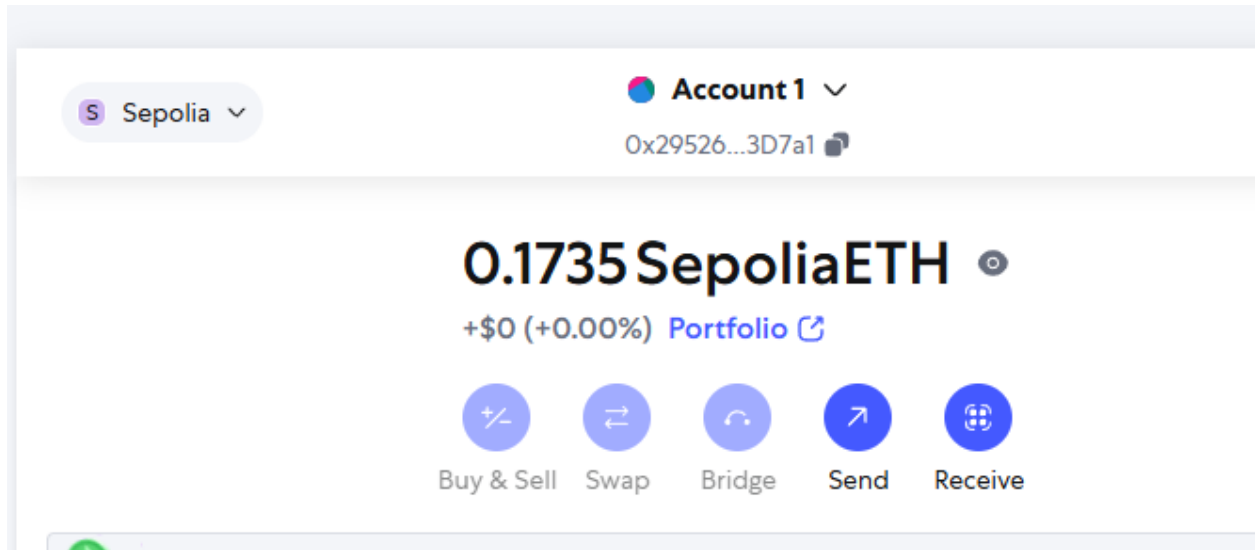
☒ ON

Manage Institutional Wallets

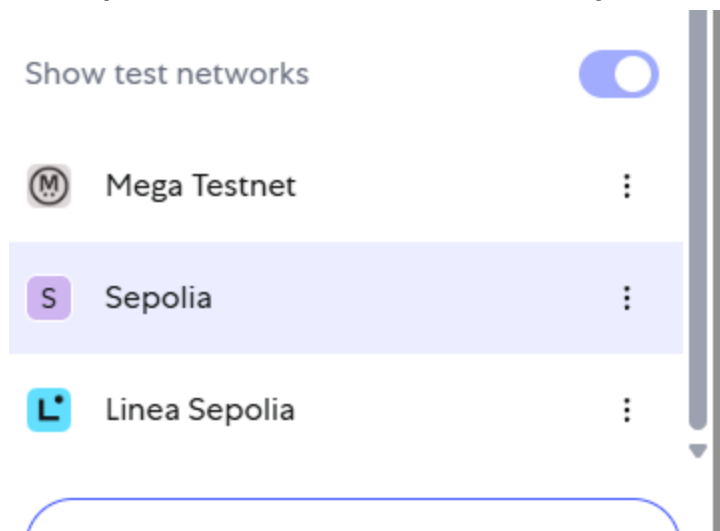
Turn this on to enable institutional wallets.

☐ OFF

4. Open the network dropdown and select **Sepolia**.



Checkpoint: MetaMask network shows **Sepolia**.



Task 1.2: Get Sepolia ETH from a Faucet

Use any faucet you prefer. Two common options:

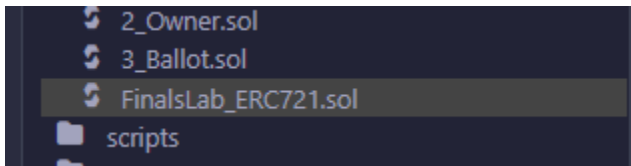
- MetaMask testnet faucet
 - Chainlink Sepolia faucet
1. Copy your MetaMask wallet address.
 2. Request test ETH from the faucet.
 3. Wait until the balance updates.

Checkpoint: You have at least **0.2 Sepolia ETH** (enough for deploy + multiple mint tests).

Part 2 — Remix Setup: Create the Contract File

Task 2.1: Open Remix and Create a File

1. Go to **remix.ethereum.org**
2. In **File Explorer**, click **New File**
3. Name it:



4. Paste the contract code (exactly as given).

Checkpoint: File saved and visible in the explorer.

Part 3 — Dependencies: OpenZeppelin Imports

Task 3.1: Resolve OpenZeppelin Imports

Your contract uses:

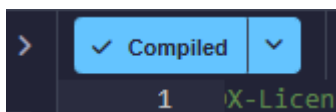
- Ownable.sol
- ERC721URIStorage.sol

```
solidity ^0.8.0;

"@openzeppelin/contracts/access/Ownable.sol";
"@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";

contract SimpleMintContract is ERC721URIStorage, Ownable {
```

1. Click **Solidity Compiler** tab.



2. Click **Compile** once.
3. If you see import errors (OpenZeppelin not found):

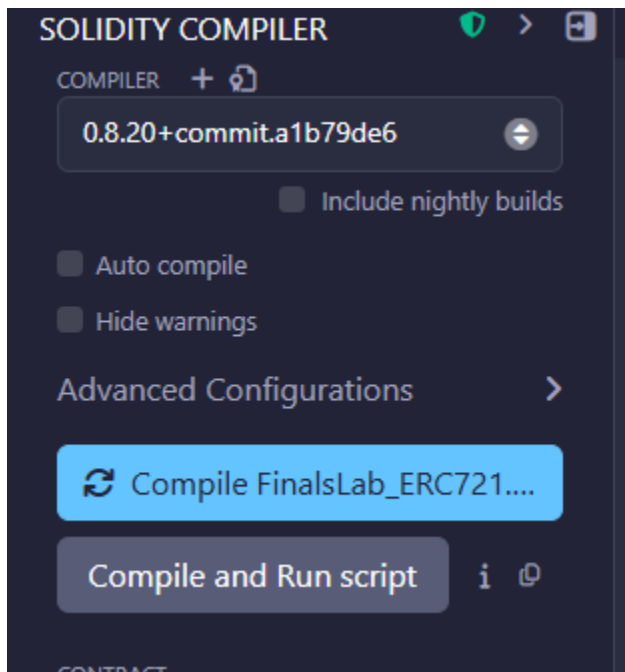
- Go to **File Explorer**
- Look for **Dependencies / Remix Packages**
- Add/install **OpenZeppelin Contracts**

Checkpoint: The contract compiles without import errors.

Part 4 — Compile the Contract

Task 4.1: Compile Settings

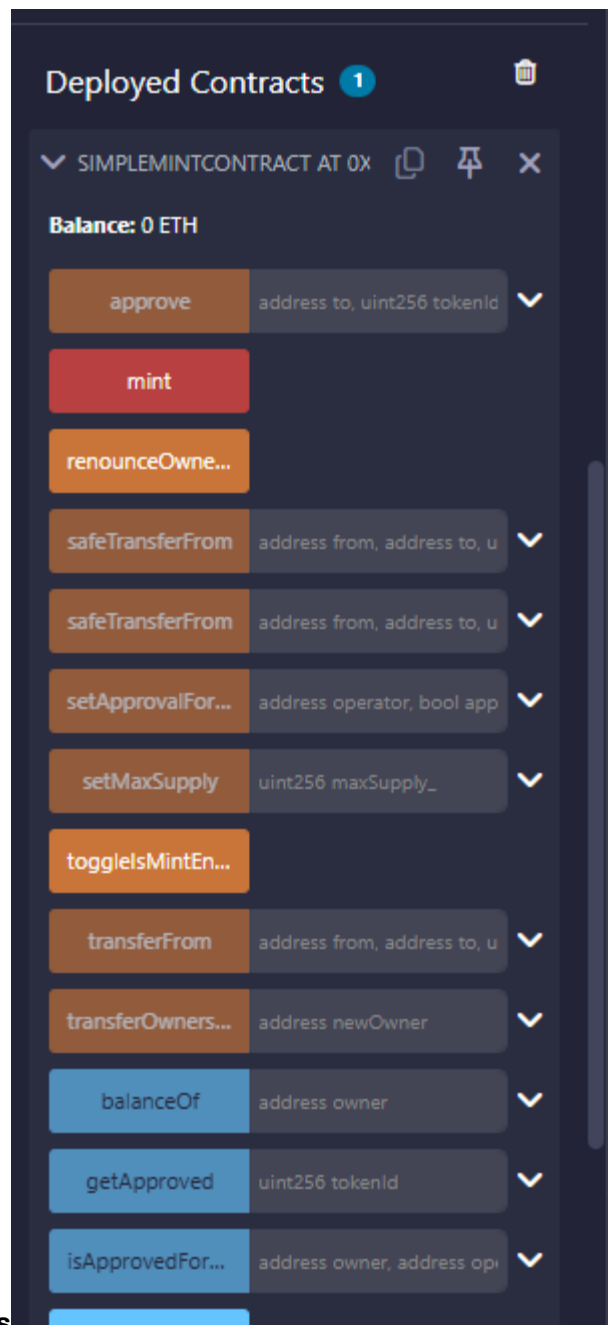
1. In **Solidity Compiler**:
 - Set compiler version to a **0.8.x** version (example: 0.8.20+)
 - Ensure it matches pragma solidity ^0.8.0;
2. Click: **Compile SimpleMintContract.sol**



Checkpoint: “Compilation successful” appears and no red errors show.

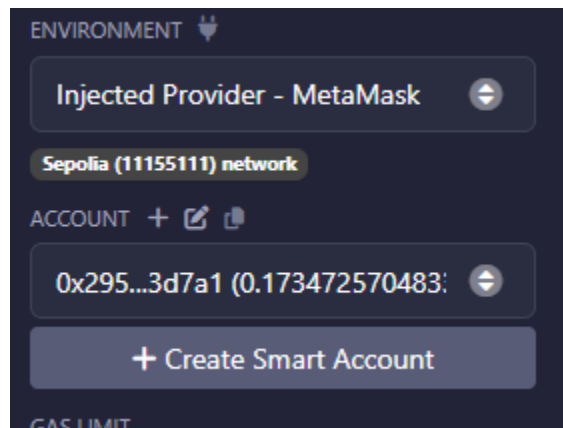
Part 5 — Connect Remix to MetaMask (Injected Provider)

Task 5.1: Use Injected Provider



1. Go to **Deploy & Run Transactions**
2. Under **Environment**, select:

- **Injected Provider – MetaMask** (or “Injected Web3” depending on UI)



3. MetaMask will pop up:
 - Approve connection
4. Confirm MetaMask is still on **Sepolia**

Checkpoint: Remix shows your MetaMask address under “Account”.

Part 6 — Deploy the ERC-721 Contract

Task 6.1: Deploy

1. In Remix **Deploy** section:
 - Contract: SimpleMintContract
2. Click **Deploy**



Account 1
Sepolia



Deploy a contract

This site wants you to deploy a contract

Estimated changes ?

No changes

Request from ?

remix.ethereum.org

Network fee ?

 0.0036 s SepoliaETH

Speed

 Market ~16 sec

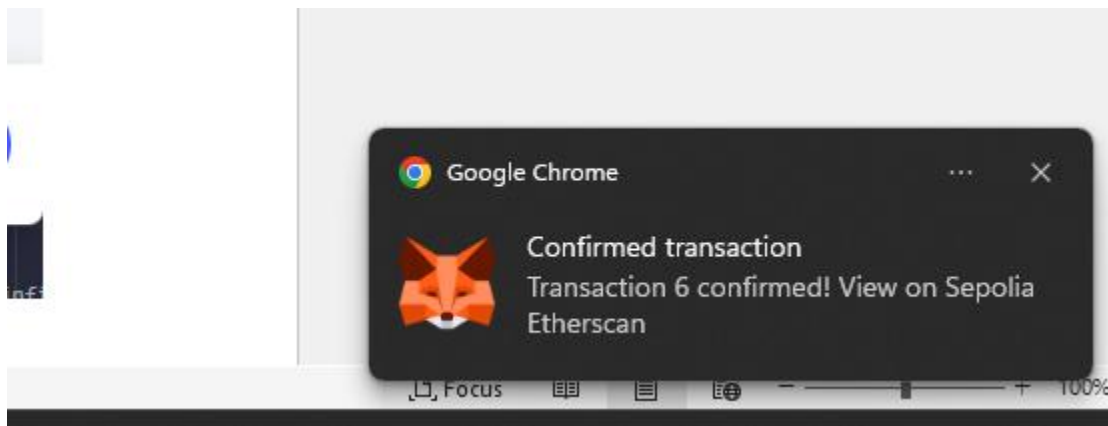
Cancel

Confirm

```
256 maxSupply } external onlyOwner {  
    info
```

3. MetaMask pops up → Confirm

Checkpoint: Contract appears under **Deployed Contracts** in Remix.



Task 6.2: Validate Initial Contract State (Read Calls)

In **Deployed Contracts**, click these **read** functions:

Function	Expected Value
owner()	your MetaMask address
isMintEnabled()	false
maxSupply()	2
totalSupply()	0
mintAmount()	50000000000000000 wei (0.05 ETH)

isApprovedFor...

address owner, address opi

▼

isMintEnabled

0: bool: false

maxSupply

0: uint256: 2

mintAmount

0: uint256: 500000000000000000

mintedWallets

address

▼

name

owner

0: address: 0x29526e6A3153a24bc8eDA6057e64834A2893D7a1

ownerOf

uint256 tokenId

▼

supportsInterfa...

bytes4 interfacId

▼

symbol

tokenURI

uint256 tokenId

▼

totalSupply

0: uint256: 0

Low level interactions

i

CALLDATA

Transact

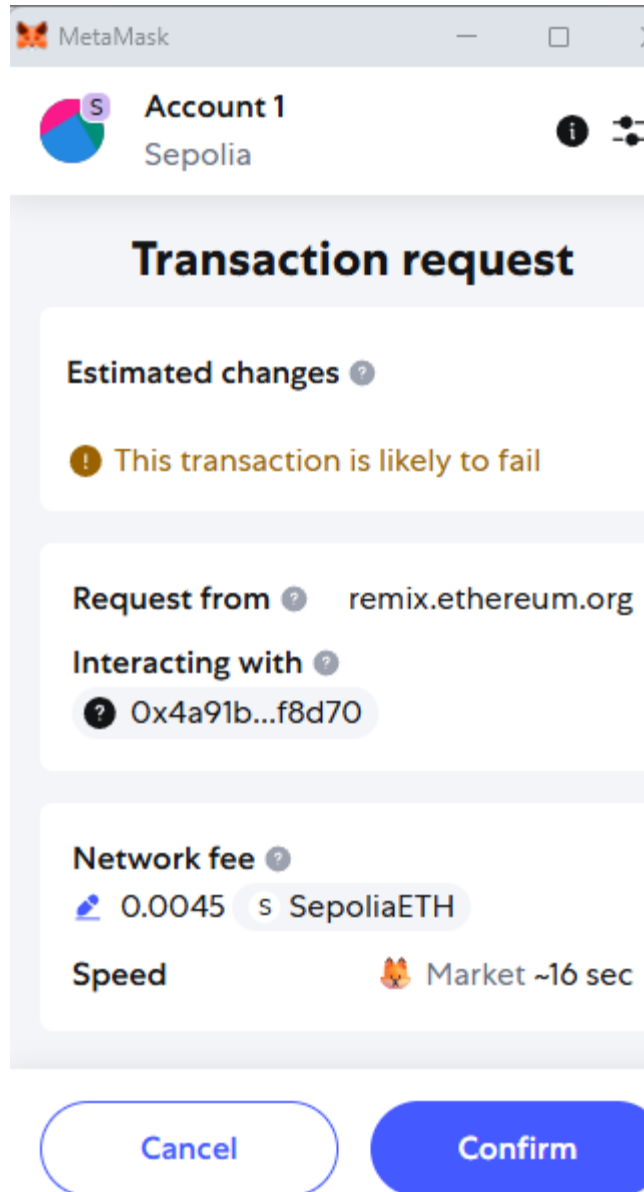
Checkpoint: All values match.

Part 7 — Functional Test Cases (Minting Rules)

Test Case 1 — Mint while Disabled (Should Fail)

Task 7.1: Attempt Mint (Expect Revert)

1. Expand mint()
2. In Remix top bar, set:
 - **Value:** 0.05
 - **Unit:** ether



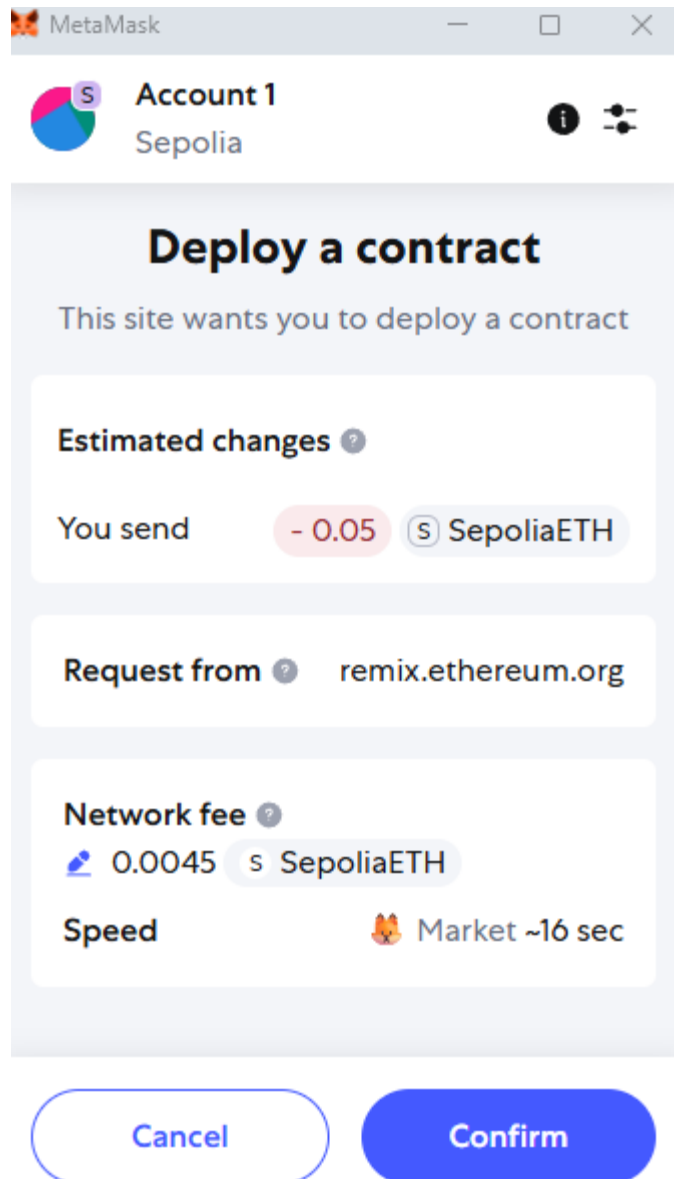
3. Click `mint()`.
4. Confirm transaction if MetaMask asks (it may fail before prompting depending on UI)

Expected Result: Transaction fails with:

- "Minting not enabled"

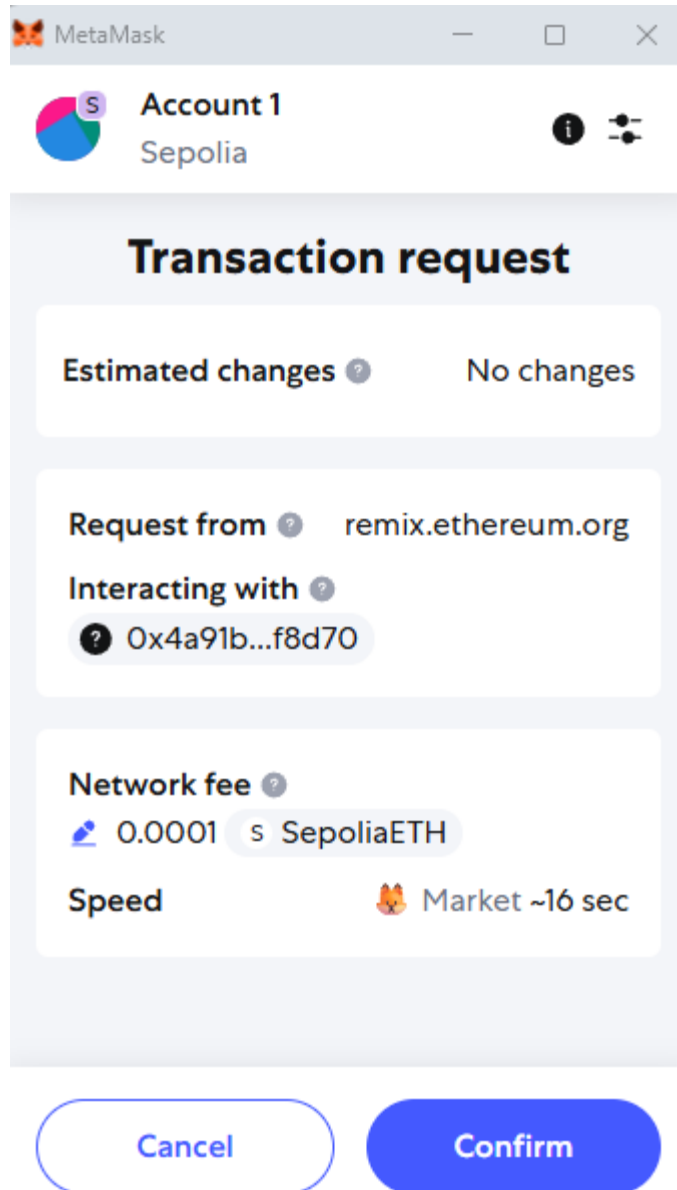
Record: Write the revert message in your lab notes.

Test Case 2 — Enable Minting (Owner Only)



Task 7.2: Toggle Minting

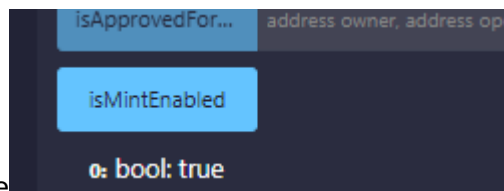
1. Click toggleisMintEnabled()
2. Confirm in MetaMask



3. Call `isMintEnabled()` again

Expected Result:

- `isMintEnabled()` → true



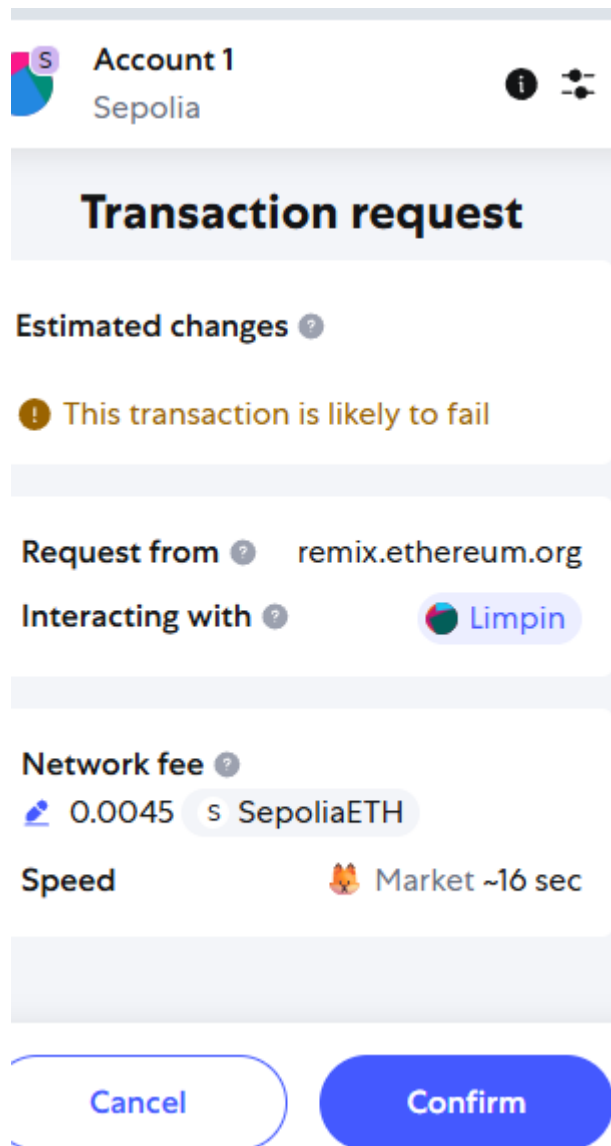
Test Case 3 — Successful Mint (Correct Payment)

Task 7.3: Mint First NFT

1. Set **Value = 0.05 ether**
2. Click mint()
3. Confirm MetaMask

Expected Result (after mined):

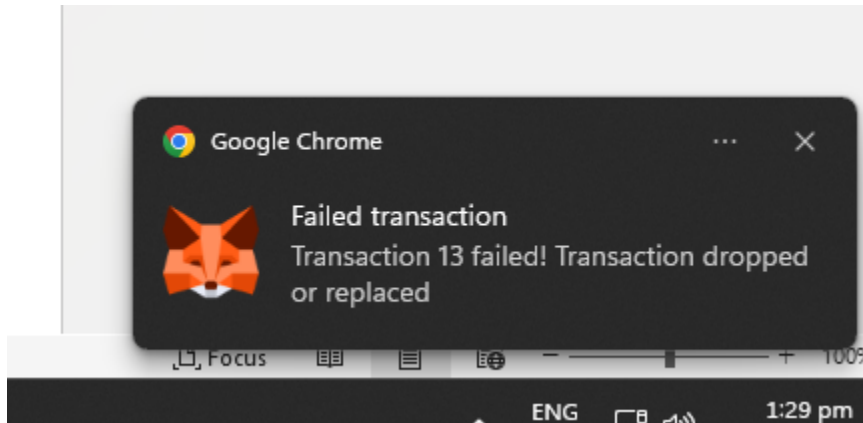
- totalSupply() → 1



- mintedWallets(yourAddress) → 1

Record: Transaction hash + updated values.

Test Case 4 — Mint Again Same Wallet (Should Fail)



Task 7.4: Attempt 2nd Mint from Same Wallet

1. Set **Value = 0.05 ether**
2. Click mint()

Expected Result: Revert message: "**Exceeds max per wallet**"

Test Case 5 — Max Supply Behavior (Need 2nd Wallet)

Because:

- maxSupply = 2
 - each wallet can only mint 1
-

Task 7.5: Mint Using Another MetaMask Account

1. In MetaMask, switch to **Account 2**
2. Ensure Account 2 has Sepolia ETH
3. Go back to Remix (Injected Provider automatically uses active account)
4. Call mint() with **0.05 ether**

Expected Result: totalSupply() → 2

Test Case 6 — Sold Out (Should Fail)

Task 7.6: Attempt Another Mint (Any Wallet)

Try minting again with any account.

Expected Result: Revert message: **"Sold out"**

Part 8 — Common Gotchas (Troubleshooting Checklist)

Issue A: Wrong Network

Symptom: Transactions fail / contract not visible / wrong chain

- Fix: Ensure MetaMask network is **Sepolia**
- Reminder: Injected Provider follows MetaMask network

Issue B: Wrong Value Sent

Symptom: Revert “Wrong value”

- Fix: Your contract requires:
 - `msg.value == mintAmount` exactly
- Use **0.05 ether** or **50000000000000000 wei**

Issue C: ERC721URIStorage “Unused”

Observation: You imported URI storage but did not set token URIs.

- Result: You can mint NFTs, but **they won’t have metadata URIs** unless you implement `_setTokenURI`.
-

Part 9 — Post-Lab Questions (Assessment)

Answer briefly:

1. Why did `mint()` fail before enabling minting?
 - **`mint()` failed because the `isMintEnabled` flag was set to false, which prevents minting. The function checks this condition and reverts if minting is not enabled.**
2. What specific line enforces “1 mint per wallet”?

This line enforces the rule that each wallet can only mint once:

`require(mintedWallets[msg.sender] < 1, "Exceeds max per wallet");`

3. What happens if you send 0.051 ETH?
- If you send 0.051 ETH, the transaction will fail. The contract expects exactly 0.05 ETH, so anything more or less will trigger an error and prevent the minting.
 4. Why do you need a second wallet to reach maxSupply = 2?
- You need a second wallet because the contract allows only one mint per wallet. With a maxSupply of 2, once the first wallet mints, the second wallet is required to mint the second token.
 5. What is the value of mintAmount in wei?
- The value of mintAmount is 50000000000000000 wei, which is the equivalent of 0.05 ETH.
-

Part 10 — Optional Extension (Metadata Upgrade Challenge)

Challenge Goal

Upgrade the contract so each minted token has a metadata URI.

Suggested approach

- Modify mint() to accept a URI:
 - mint(string memory uri) external payable
- After _safeMint, call:
 - _setTokenURI(tokenId, uri);

Success criteria:

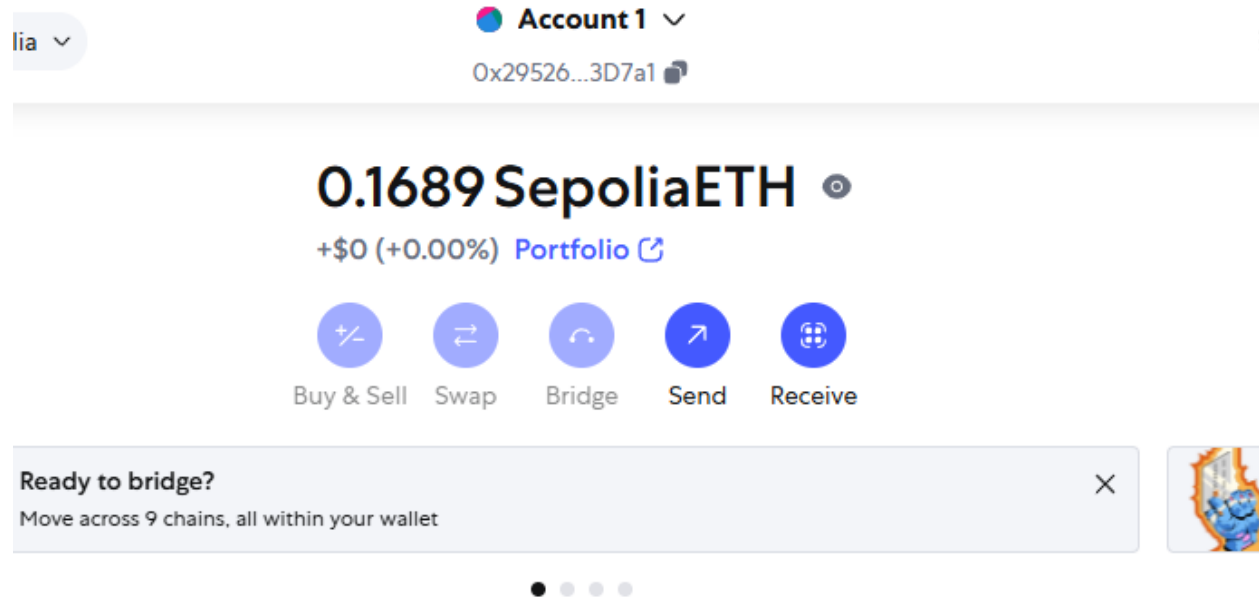
- After minting, calling tokenURI(tokenId) returns the URI you provided.
-

Student Output Checklist (What to Submit)

Screenshots of the following:

- 1.

1. MetaMask showing Sepolia network + balance



2. Remix deployed contract
3. Successful mint transaction (1st wallet)
4. Revert: "Minting not enabled"
5. Revert: "Exceeds max per wallet"
6. Successful mint transaction (2nd wallet)
7. Revert: "Sold out"

[Previous](#)